

ASSIGNMENT 4

Clustered Smart Home Alarm System

Analisi dei requisiti

Il progetto richiede l'estensione di un sistema Smart Home Alarm centralizzato verso un'architettura distribuita basata su Apache Pekko Cluster. Il sistema deve operare su una rete di nodi clusterizzati.

È richiesto un minimo di 3 nodi, e la comunicazione deve avvenire esclusivamente tramite messaggi degli attori. Abbiamo scelto di implementare 4 nodi separati:

- La control unit (SmartHomeAlarmGuardian),
- Sensori (Sensor)
- Il tastierino (UserInteraction)
- La sirena (Alarm).

Il sistema deve garantire la location transparency sfruttando meccanismi di Service Discovery (Pekko Receptionist e Group Routers).

I sensori devono operare come attori distribuiti, con identificatori univoci (UUID) e gli eventi devono essere recapitati al nodo di controllo. Deve essere preservata la logica della Macchina a Stati Finiti (Entry Delay, Exit Delay). In caso di crash e successivo riavvio del nodo centrale, l'entità perde il proprio stato in memoria. Pertanto, per evitare vulnerabilità di sicurezza (es. assumere erroneamente che la casa sia disarmata), il sistema deve avviarsi obbligatoriamente in uno stato di Recovery. Durante il Recovery, gli eventi dei sensori devono essere tracciati o ignorati, finché l'utente non inserisce il PIN corretto per sbloccare la centralina nello stato Disarmed

Architettura

Docker containers:

Per simulare l'esecuzione dei 4 nodi in ambienti differenti come sarebbero in un reale sistema distribuito, abbiamo usato docker al fine di creare un container per ciascun gruppo di attori. Inoltre, questa scelta rende molto più semplice l'esecuzione contemporanea del programma (README del progetto per maggiori informazioni). La control unit agisce come nodo "seed" principale del cluster, esponendo la porta 1600 a cui fanno riferimento tutti gli altri nodi per l'ingresso nella rete. I nodi periferici si collegano al seed utilizzando porte assegnate in modo incrementale a partire dalla 2551.

Per gestire la failure del nodo senza l'utilizzo di database esterni, la Control Unit verifica (all'avvio nel suo setup) l'esistenza di un file guardian-started.lock all'interno del file system effimero del container. Se il file è assente, il sistema lo crea ed entra nello stato iniziale disarmed; se è già presente, il nodo si rende conto di aver subito un crash ed entra nello stato di recovery.

Receptionist:

Rispetto alla versione non distribuita il primo cambiamento è l'implementazione di un'entità il cui compito è fare da tramite tra i diversi nodi presenti. Processo esecutivo:

- Creazione di una serviceKey: in ciascun nodo, per essere identificato univocamente, viene creata una chiave (es. guardianServiceKey).
- Sottoscrizione al Receptionist: una volta creata la chiave, il nodo viene inserito nel "registro" globale inviando un comando di Receptionist.Register, rendendosi così visibile all'intero cluster
- Invio e ricezione di messaggi: nella versione distribuita, la comunicazione non avviene tramite il passaggio esplicito di ActorRef. I nodi mittenti utilizzano i Group Routers (Routers.group(chiave_servizio)). Questo garantisce che, se il nodo destinatario (per esempio la Control Unit) va offline temporaneamente per un riavvio, il Router del mittente non genera eccezioni fatali, ma converte i messaggi in dead letters, ripristinando il normale flusso di esecuzione non appena il destinatario si registra al Receptionist con il suo nuovo id.

Gestione dei Sensori e identificatori univoci:

Abbiamo scelto di assegnare a ciascun sensore un identificatore univoco costante (sensorId generato tramite UUID) al momento della sua creazione. Infatti, in una rete distribuita, un sensore potrebbe disconnettersi e ricollegarsi. La control unit deve essere quindi in grado di riconoscere la tipologia di sensore specifica. Quando il container sensor avvia i suoi attori (tramite Behaviors.setup), a ciascuno di essi viene assegnato l'identificatore univoco che rimane costante per tutta la vita del sensore.

Anziché creare più ServiceKey diverse sul receptionist per distinguere, ad esempio, il piano terra dal primo piano, abbiamo inserito l'identità del sensore come parametro in input nel messaggio che viene spedito.

Il sensore prende il dato e lo manda alla Control Unit (router ! DetectedMovement(...)).

Quando il Guardian riceve il messaggio, apre il pacchetto, legge il sensorId scritto al suo interno e lo registra. Tramite questa scelta implementativa, potremmo scalare ed estendere il nostro progetto in qualsiasi momento con facilità, aggiungendo altri sensori al container, e la control unit li riconoscerebbe tutti perfettamente, senza dover modificare alcuna riga di codice nel suo apply().

Collaudo

Per verificare la reale tolleranza ai guasti, abbiamo condotto dei test mirati direttamente sull'infrastruttura in Docker, analizzando il comportamento dei cluster. Mentre il sistema era attivo, abbiamo forzato il riavvio del nodo centrale, riavviando direttamente tramite il comando "docker restart ..." e fermandolo e riattivandolo tramite "docker stop ..." e "docker start ...". Osservando i log di sistema, abbiamo confermato che il nodo esce dal cluster, e al successivo riavvio la logica di lettura del file guardian-started.lock si attiva in maniera corretta, forzando la transizione nello stato di Recovery. Il sistema richiede, a questo punto, l'inserimento del PIN predefinito (1234) tramite il nodo keypad per sbloccare la centralina e tornare allo stato disarmed. A quel punto, sarà necessario reinserire il PIN predefinito e scegliere poi la modalità (full, giorno o notte).

Abbiamo anche testato lo spegnimento forzato dei nodi periferici (come alarm o keypad). La Control Unit rimane stabile e mantiene il suo stato logico in memoria, dimostrando che il disaccoppiamento garantito dal pattern di Service Discovery previene qualsiasi eccezione fatale legata all'assenza improvvisa degli attori dipendenti.